



Mobile Application Development (2/68)

Fri. 13.00 – 16.00

Online: MS Team (02204472 Mobile App. (2/68))

Objectives



Introduction to mobile platforms and how to program these platforms



Introduction to programming for mobile platforms



Presentation of methods for developing applications for mobile devices

Grading

- Assignments 40%
- Midterm 30%
- Final 30%

Android, iOS, Windows

Platform	Language	IDE	Notes
Android	Java, c , c++	Android Studio	Open Source,
iOS	Swift, Objective-C	X Code	-Apple machine only -Requires Apple Developer Account
Windows	C#	Visual Studio	



	Flutter	React Native
What is it?	A portable UI toolkit for building natively-compiled apps across mobile, web, and desktop* from a single codebase	A framework for building native applications using React
Official release	December 2018, Google I/O	March 2015, F8 Conference
Created by	Google	Facebook
Free and open source	Yes	Yes
Programming language	Dart	JavaScript
Popularity	120,000 Stars on Github (May 2021)	95,300 stars on Github (May 2021)
Hot Reload	Yes	Yes
Native performance	Great	Great



React Native



iOS



Android

React Native lets you build mobile apps using only JavaScript. It uses the same design as React, letting you compose a rich mobile UI from declarative components. With React Native, you don't build a mobile web app, an HTML5 app, or a hybrid app; you build a real mobile app that's indistinguishable from an app built using Objective-C or Java. React Native uses the same fundamental UI building blocks as regular iOS and Android apps. You just put those building blocks together using JavaScript and React.

React Native Features

React – This is a Framework for building web and mobile apps using JavaScript.

Native – You can use native components controlled by JavaScript.

Platforms – React Native supports iOS and Android platform.

React Native Advantages

01

JavaScript – You can use the existing JavaScript knowledge to build native mobile apps.

02

Code Sharing – You can share most of your code on different platforms

03

Platforms – The community around React and React Native is large, and you will be able to find any answer you need.

Who uses React Native

Facebook

Instagram

Walmart

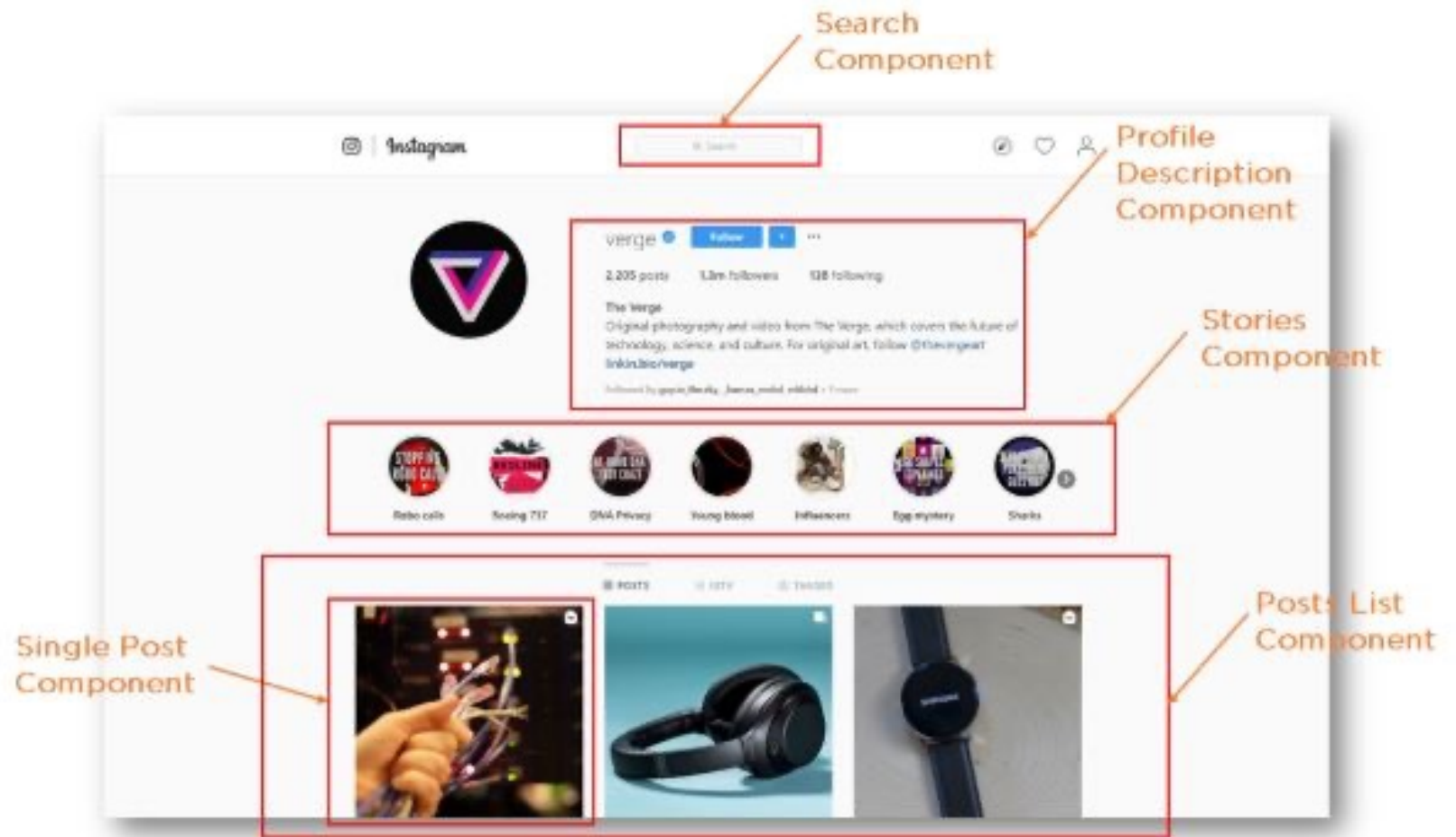
Tesla

Bloomberg

Baidu

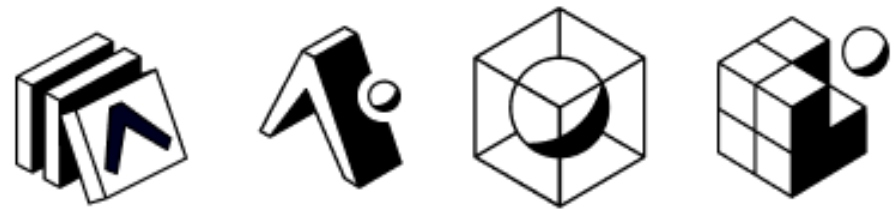
Skype

How React works!

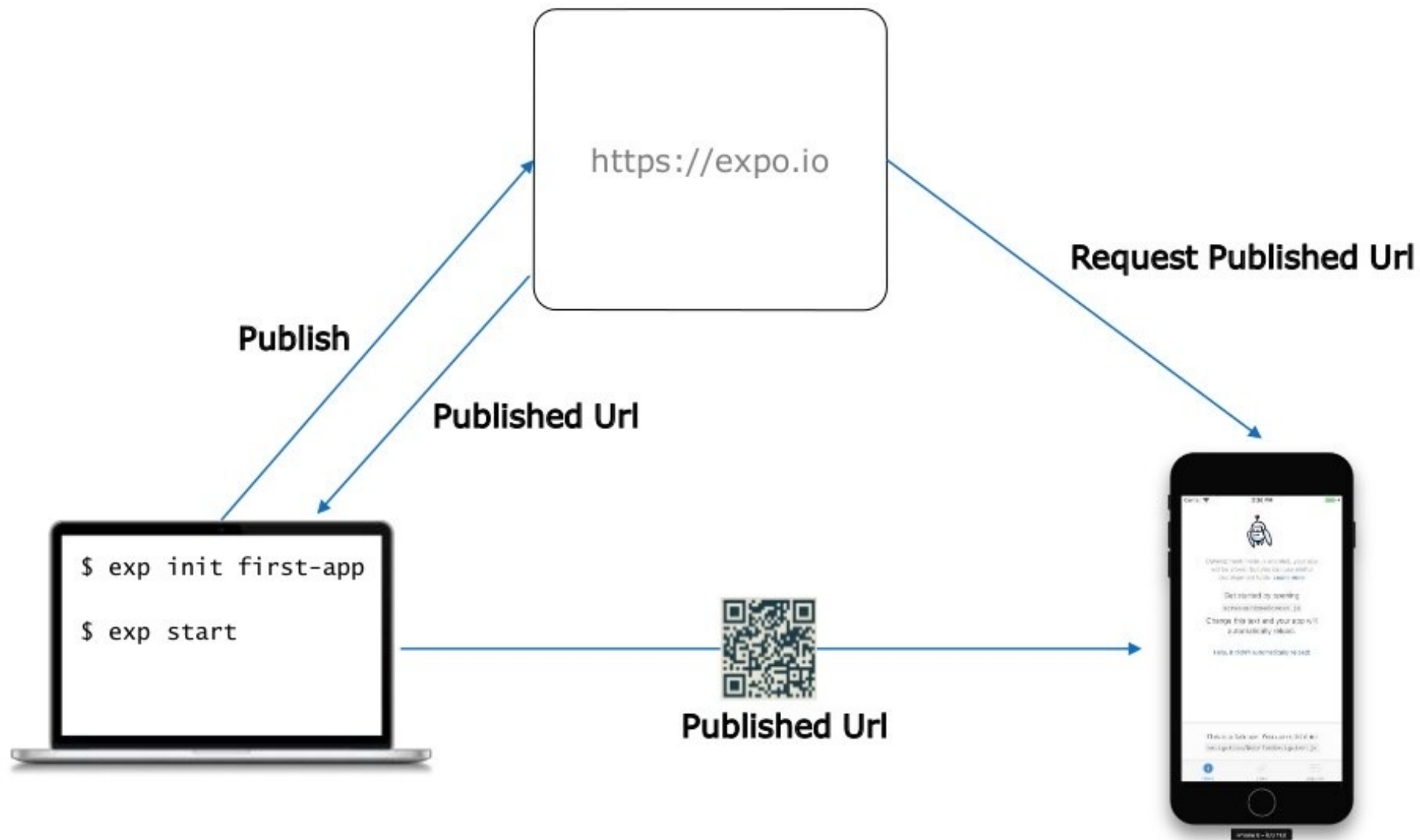


Expo is a framework and a platform for universal React applications. It is a set of tools and services built around React Native and native platforms that help you develop, build, deploy, and quickly iterate on iOS, Android, and web apps from the same JavaScript/TypeScript codebase.

^ Expo



How to use



Installation

- Node JS(LTS) (<https://nodejs.org/en/>)
- Visual Studio Code
- Android Studio
- Xcode (MacOS)



Checking

- `node -v`
- `npm -v`

Node.js

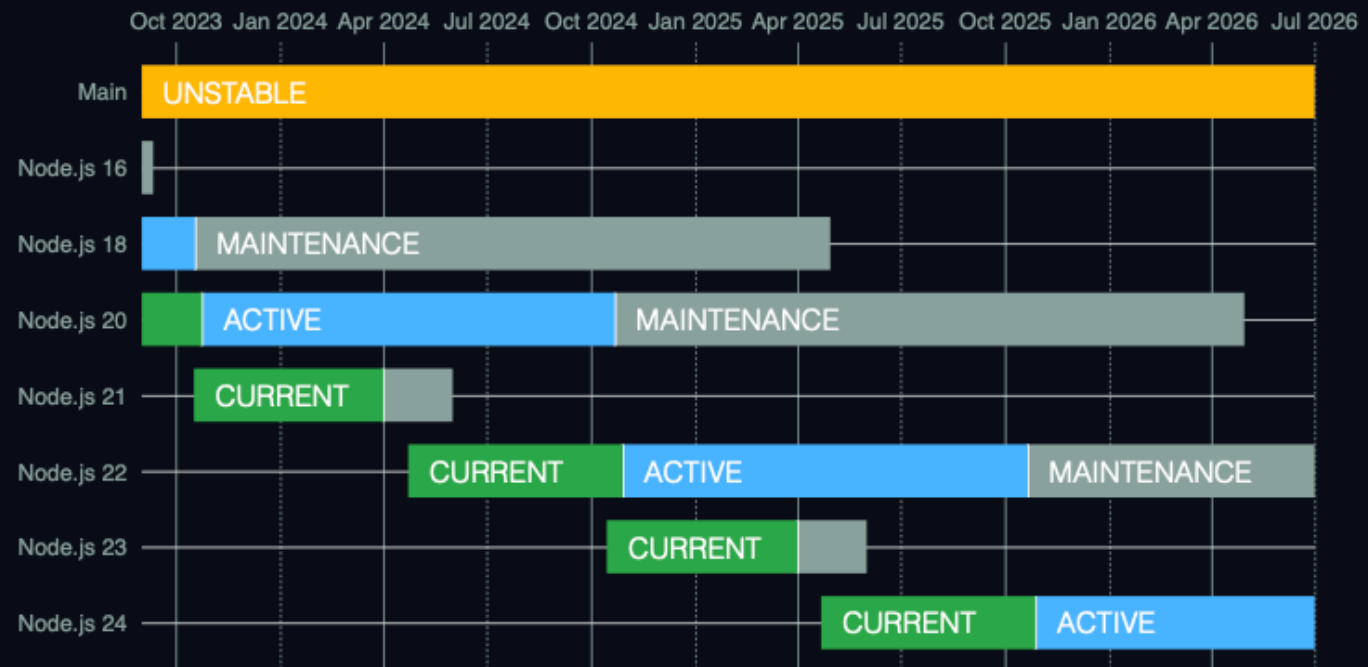
Downloads

Latest LTS Version: **20.9.0** (includes npm 10.1.0)

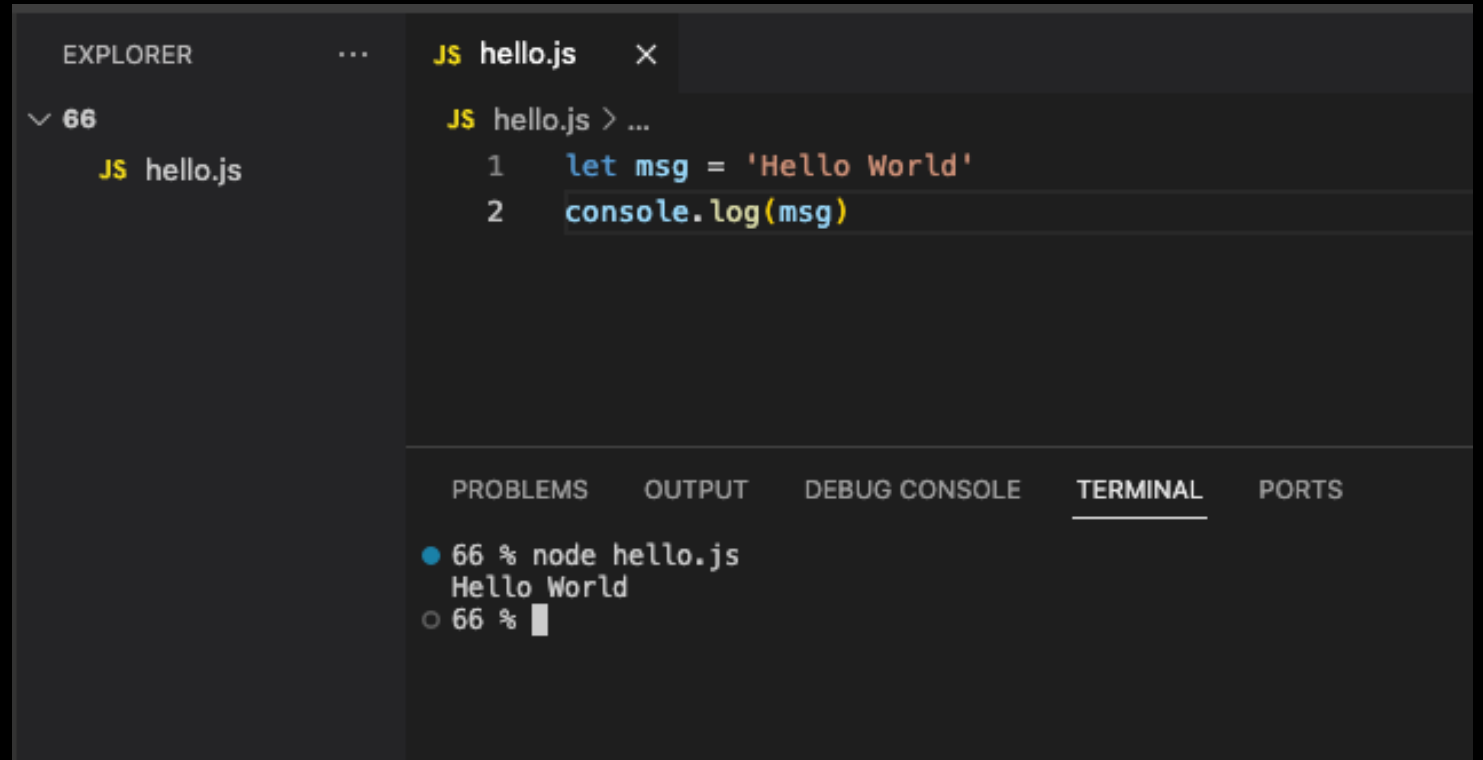
Download the Node.js source code or a pre-built installer for your platform, and start developing today.

LTS Recommended For Most Users	Current Latest Features	
 Windows Installer node-v20.9.0-x86.msi	 macOS Installer node-v20.9.0.pkg	 Source Code node-v20.9.0.tar.gz

Release Schedule



Simplest Node.js application

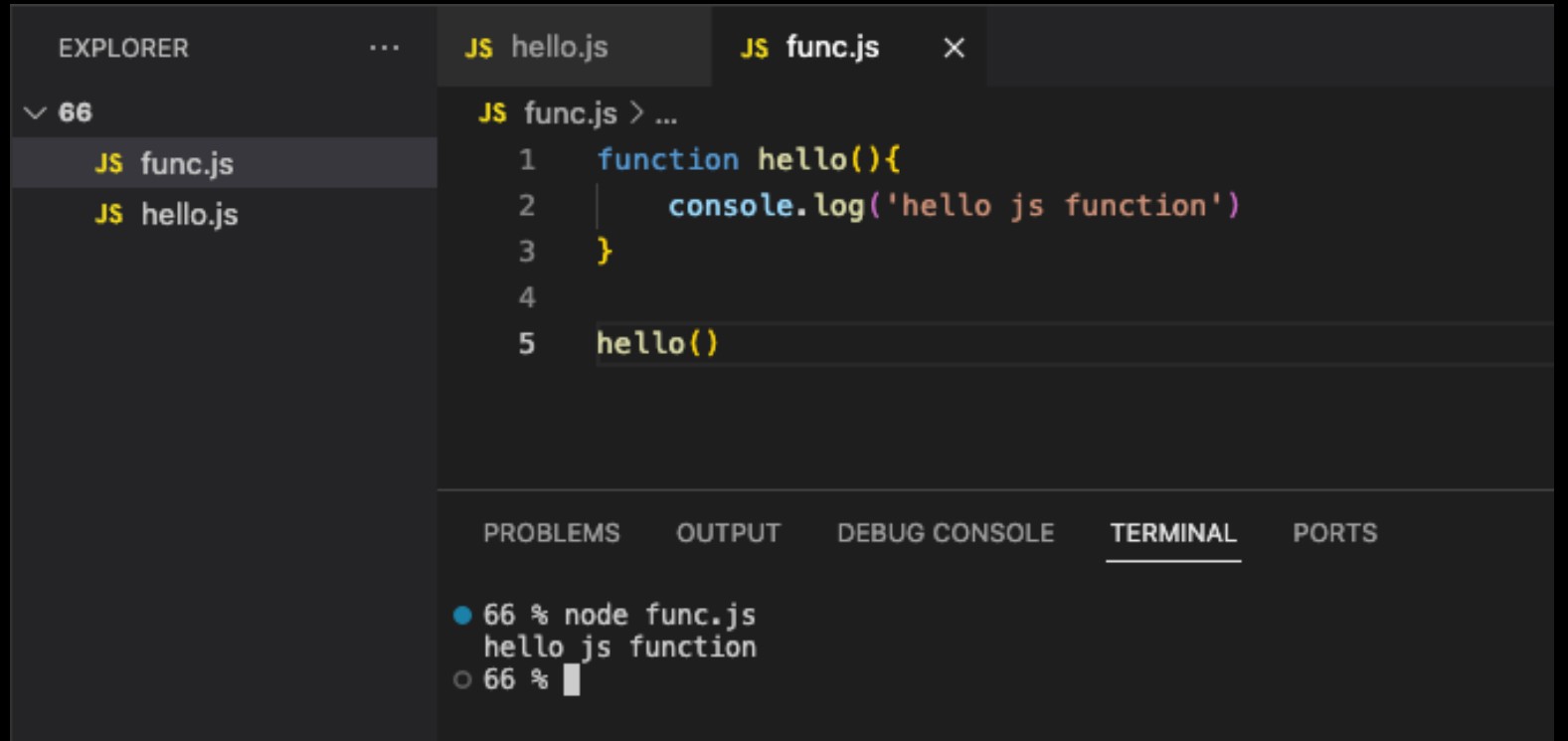


The screenshot shows the Visual Studio Code interface with a dark theme. On the left, the Explorer sidebar shows a folder named '66' containing a file 'hello.js'. The main editor area has a tab for 'hello.js' with the following code:

```
JS hello.js > ...  
1 let msg = 'Hello World'  
2 console.log(msg)
```

Below the editor, the bottom panel shows the 'TERMINAL' tab selected. It displays the command '66 % node hello.js' and its output 'Hello World'.

JavaScript Function



The image shows a Visual Studio Code editor window with two tabs: `JS hello.js` and `JS func.js`. The `func.js` tab is active, displaying the following JavaScript code:

```
1 function hello(){  
2     console.log('hello js function')  
3 }  
4  
5 hello()
```

The Explorer sidebar on the left shows a folder named `66` containing `func.js` and `hello.js`. At the bottom, the TERMINAL panel shows the command `node func.js` being executed, which results in the output `hello js function`.

JavaScript extensions in VS Code

Using tags: "tag:javascript"

 Prettier - Code format... esbenp ⬇ 38.5M	 IntelliCode VisualStudioExptl ⬇ 36.1M	 JavaScript (ES6) code ... xabikos ⬇ 13.9M	 Babel JavaScript mgmcdermott ⬇ 2.7M
 Node.js Modules Intel... leizongmin ⬇ 1.4M	 Node.js Extension Pack waderyan ⬇ 978.4K	 Node.js Exec miramac ⬇ 585.2K	 Node.js Assertion Sni... jaymorrow ⬇ 51.3K

JavaScript Functions

```
function hello() {  
    console.log('Hello Function!')  
}
```

```
const helloArrow = () => {  
    console.log('Hello Arrow Function!')  
}
```

```
hello()  
helloArrow()
```

```
function square(n) {  
    return n*n  
}
```

```
const squareA = (m) => {  
    return m*m  
}
```

```
console.log(square(4))  
console.log(squareA(5))
```

```
function add(x, y) {  
    return x + y  
}
```

```
const addA = (a, b) => {  
    return a + b  
}
```

```
let res = add(2, 5)  
let ret = addA(6, 3)  
console.log(res)  
console.log(ret)
```

```
function isEven(num) {  
    return num % 2 === 0  
}
```

```
const isOdd = (n) => {  
    return n % 2 === 1  
}
```

```
console.log(isEven(25)) //false  
console.log(isOdd(25)) //true
```

```
function factorial(n) {  
    if(n === 0){  
        return 1  
    }  
    return n * factorial(n-1)  
}
```

```
console.log(factorial(6)) //720
```

```
const absValue = (n) => {  
    if(n < 0) {  
        return -n  
    }  
    return n  
}
```

```
console.log(absValue(-15))  
console.log(absValue(7))
```


// Old way

```
const sum = () => {  
  let result = 0  
  for(let i=0; i<arguments.length; i++){  
    result += arguments[i]  
  }  
  return result  
}
```

```
console.log(sum(1))  
console.log(sum(1,2))  
console.log(sum(1,2,3,4))  
console.log(sum(1,3,5,7,9))
```

// New way using REST parameter

```
const sum = (...args) => {  
  let result = 0  
  for(let i=0; i<args.length; i++){  
    result += args[i]  
  }  
  return result  
}
```

```
const sum = (...args) => {  
  let result = 0  
  result = args.reduce((current, prev) => {  
    return current + prev  
  })  
  return result  
}
```

// A function taking 'function' as a parameter

```
function dispatch(fn) {  
    fn()  
}
```

//METHOD 1:

```
var fn = () => {console.log('Hello!')}  
dispatch(fn)
```

//METHOD 2:

```
dispatch(function() {  
    console.log('Hello!')  
})
```

//METHOD 3:

```
dispatch (() => {console.log('Hello!')})
```

// A function taking 'function' as a parameter

```
function dispatch(fn) {  
    return fn('Hello!')  
}
```

```
var fn = (m) => {  
    console.log(m)  
}
```

```
dispatch(fn)
```

```
function patch(fn) {  
    let res = fn(5,6)  
    console.log(res)  
}
```

```
var add = (a, b) => {  
    return a + b  
}
```

```
patch(add)
```